

BLOCKCHAIN-BASED FREELANCE PAYMENT ESCROW DApp

PROJECT REPORT

1. Abstract

The Blockchain-Based Freelance Payment Escrow DApp is a decentralized application for managing freelance project payments through Solidity smart contracts. Client funds are locked in blockchain escrow and released, refunded, or distributed according to predefined rules. The system provides separate Client, Freelancer, and Arbitrator dashboards and supports milestone creation, custom ETH amounts, escrow funding, work progress, payment release, refunds, disputes, arbitrator settlement, transaction activity, milestone history, and freelancer earnings.

2. Introduction

Freelance projects require trust between clients and freelancers. Clients want assurance that their funds are protected until the agreed work conditions are met, while freelancers need confidence that they will receive payment after completing the work.

Traditional freelance platforms often depend on centralized intermediaries to hold funds and manage disputes. This project introduces a programmable blockchain escrow model where smart-contract rules control the payment lifecycle, providing a transparent and structured process for funding, milestone completion, payment release, refunds, and dispute resolution.

3. Problem Statement

Freelance payment workflows can face several challenges, including lack of trust between clients and freelancers, payment delays, centralized escrow systems, manual dispute resolution, platform dependency, limited transparency, and difficulties in tracking milestone-based payments.

The proposed project addresses these challenges through a programmable smart-contract escrow mechanism. The smart contract securely controls the funds and validates payment operations according to predefined conditions, enabling a more transparent and structured process for milestone payments, refunds, and dispute resolution.

4. Objectives

- Build a decentralized escrow mechanism for freelance payments.
- Support milestone-based project and payment management.
- Allow clients to set custom ETH amounts and deadlines.
- Lock client funds in the smart contract before payment release.
- Allow freelancers to start and submit work.

- Support payment release after client approval.
- Provide eligible refund functionality.
- Provide dispute raising and arbitrator resolution.
- Provide role-specific dashboards and access control.
- Provide transaction, milestone, payment, and earnings visibility.
- Demonstrate the complete workflow safely on a local blockchain.

5. System Participants

Participant	Responsibilities	Access
Client	Creates milestones, sets amounts/deadlines, funds escrow, reviews work, releases payment, requests refunds, raises disputes, views history.	Client Dashboard
Freelancer	Views assigned milestones, starts work, submits work/proof, receives payments, raises disputes, views earnings and payment history.	Freelancer Dashboard
Arbitrator	Reviews disputed milestones, sets freelancer payout and client refund, resolves disputes and distributes escrow funds.	Arbitrator Dashboard
Smart Contract	Locks, releases, refunds and settles funds according to contract rules.	Blockchain Layer

6. Features

6.1 Client

1. MetaMask Connection: The client connects the DApp using MetaMask. The wallet address identifies the client and authorizes client-specific transactions.

2. Create Milestones: The client can create multiple milestones for a freelance project and define the work associated with each milestone.

3. Custom ETH Amount: The client can set a different ETH payment amount for each milestone based on the work involved.

4. Deadline Selection: The client can set a deadline for each milestone to define the expected completion period.

5. Fund Escrow: The client deposits the required ETH into the smart contract. The funds remain locked until the milestone is settled.

6. Track Progress

The client can view the current status of milestones, such as **Created, Funded, In Progress, Submitted, Completed, Disputed, or Refunded**.

7. Review Submitted Work: The client can review the work submitted by the freelancer before approving the payment.

8. Approve and Release Payment: After accepting the work, the client approves the milestone. The smart contract releases the payment to the freelancer.

9. Request Eligible Refund: If refund conditions are satisfied, the client can request a refund of the escrowed amount.

10. Raise Disputes: If there is a disagreement with the freelancer, the client can raise a dispute for arbitrator resolution.

11. Milestone History: The client can view previous milestones, their amounts, statuses, and payment outcomes.

12. Project Payment Tracking: The client can track the overall payment activity, including funded, released, and refunded amounts for the project.

6.2 Freelancer

1. MetaMask Connection: The freelancer connects to the DApp using MetaMask. The wallet identifies the freelancer and authorizes transactions.

2. View Assigned Milestones: The freelancer can view the milestones assigned by the client, including the milestone amount, deadline, and status.

3. Start Work: The freelancer can start an assigned milestone after it has been funded by the client.

4. Submit Completed Work/Proof: After completing the work, the freelancer can submit the work or proof for the client to review.

5. Track Progress: The freelancer can monitor the status of assigned milestones and payment progress.

6. Raise Disputes: If a disagreement occurs, the freelancer can raise a dispute for arbitrator resolution.

7. Receive Payments: After the client approves the completed work, the smart contract releases the milestone payment to the freelancer.

8. Total ETH Earned: The freelancer dashboard displays the total ETH earned from completed milestones.

9. Payment History: The freelancer can view previous milestone payments and their transaction details.

10. Dispute Payouts: When a dispute is resolved by the arbitrator, the freelancer can view the amount received from the dispute settlement.

6.3 Arbitrator

1. Arbitrator Wallet Access: Connects the arbitrator's wallet to access dispute-related functions.

2. Review Disputed Milestone: Reviews the milestone involved in the dispute.

3. View Escrow Information: Views the milestone amount and escrow details.

4. Set Freelancer Payout: Specifies the amount to be paid to the freelancer.

5. Set Client Refund: Specifies the amount to be returned to the client.

6. Add Resolution Information: Provides details about the dispute decision.

7. Resolve Dispute: Confirms the final dispute decision through the smart contract.

8. Distribute Escrow Funds: The smart contract distributes the escrow amount according to the arbitrator's decision.

7. Normal Escrow Workflow

1. **Client creates a milestone** for the freelance project.
2. **Client sets the ETH amount and deadline** for the milestone.
3. **Client funds the escrow** by depositing the required ETH.
4. **Smart contract locks the ETH** until the milestone is settled.
5. **Freelancer starts the work** after the milestone is funded.
6. **Freelancer submits the completed work** for client review.
7. **Client reviews the submission** and checks whether the work meets the requirements.
8. **Client approves the milestone** after accepting the work.
9. **Smart contract releases the payment** according to the escrow rules.
10. **Freelancer receives the ETH** as the milestone payment.

8. Dispute Workflow

1. **Client or freelancer raises a dispute** when a disagreement occurs.
2. **Milestone enters the DISPUTED state** and normal payment processing is paused.
3. **Arbitrator reviews the escrow information** and the disputed milestone.
4. **Arbitrator decides the payout** for the freelancer and refund for the client.
5. **Smart contract distributes the funds** according to the arbitrator's decision.
6. **Freelancer receives the assigned payout** from the escrow.
7. **Client receives the assigned refund** from the remaining escrow amount.

9. Refund Workflow

1. **Milestone becomes eligible for a refund** based on the applicable refund conditions.
2. **Client requests the refund** through the DApp.
3. **Smart contract validates the request** and checks the required conditions.
4. **Escrow funds are returned to the client** after successful validation.
5. **Milestone becomes REFUNDED**, completing the refund process.

10. Milestone States

State	Meaning
CREATED	Milestone created
FUNDED	Client deposited funds
IN_PROGRESS	Freelancer started work
SUBMITTED	Freelancer submitted work
COMPLETED	Payment released
DISPUTED	Dispute raised
REFUNDED	Funds returned to client

11. Custom Milestone Amounts

Milestone amounts are not fixed. The client enters the required ETH amount for each milestone and the selected value is supplied to the smart contract during funding.

Example	Amount
Milestone 1	0.001 ETH
Milestone 2	0.005 ETH
Milestone 3	0.010 ETH
Milestone 4	0.025 ETH

12. Dispute Settlement Example

For an escrow containing 0.010 ETH:

Settlement	Amount
Freelancer Payout	0.006 ETH
Client Refund	0.004 ETH
Total Settlement	0.010 ETH

The two settlement amounts together equal the escrow amount.

13. Architecture

Layer	Technology	Purpose
Frontend	React + CSS + Vite	Dashboards and user interface
Web3	Ethers.js	Frontend-blockchain communication
Wallet	MetaMask	Wallet connection and transaction signing
Smart Contract	Solidity	Escrow, payment, refund and dispute rules
Development	Hardhat	Compilation, local blockchain, deployment and testing
Network	Ethereum-compatible blockchain	Contract execution

14. Project Structure

Blockchain-Based-Smart-Contract-Freelance-Escrow/

```

├── contracts/
│   └── FreelanceEscrow.sol
├── scripts/
│   ├── copy-abi.js
│   └── deploy.js
├── test/
│   └── FreelanceEscrow.test.js
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── abi/FreelanceEscrow.json
│   │   ├── App.jsx
│   │   └── App.css

```

```
| | | index.css
| | | main.jsx
| | | package.json
| | | vite.config.js
| | |
| | | screenshots/
| | | artifacts/
| | | cache/
| | | hardhat.config.js
| | | package.json
| | | package-lock.json
| | | .gitignore
| | | README.md
```

15. Technologies Used

- Solidity – smart contract development
- Ethereum-compatible blockchain – execution environment
- Hardhat – development, compilation and testing
- Ethers.js – blockchain interaction
- React.js – frontend
- JavaScript – application logic
- CSS – interface styling
- Vite – frontend development server
- MetaMask – wallet and transaction signing
- Node.js – JavaScript runtime
- npm – package management
- Git/GitHub – version control and repository hosting
- Remix IDE – smart contract development

16. Installation

16.1 Clone Repository

```
git clone https://github.com/shubha229/Blockchain-Based-Smart-Contract-Freelance-Escrow
```

```
cd Blockchain-Based-Smart-Contract-Freelance-Escrow
```

16.2 Backend

```
npm install
```

16.3 Frontend

```
cd frontend
```

```
npm install
```

```
cd ..
```

17. Execution Procedure

17.1 Start Hardhat

`npx hardhat node`

Keep this terminal running.

17.2 Compile

`npx hardhat compile`

Expected result: Compiled successfully.

17.3 Test

`npx hardhat test`

The documented execution records 11 passing tests covering payment release, double-payment prevention, dispute handling, arbitrator resolution, access control and refund workflow.

17.4 Deploy

`npx hardhat run scripts/deploy.js --network localhost`

Run from the project root while the Hardhat node is running.

17.5 Copy ABI

`node scripts/copy-abi.js`

ABI destination: `frontend/src/abi/FreelanceEscrow.json`

17.6 Run Frontend

`cd frontend`

`npm run dev`

Open the Vite URL, normally `http://localhost:5173`.

18. MetaMask Configuration

1. Connect MetaMask to the local Hardhat blockchain.
2. Import the required Hardhat test accounts.
3. Use the Client account for client actions.
4. Use the Freelancer account for freelancer actions.
5. Use the Arbitrator account for arbitrator actions.
6. Confirm the connected account matches the required role.

Never upload real private keys or seed phrases to GitHub.

19. Limitations

- Uses a local blockchain for development.
- Does not represent a production freelance marketplace.
- Work files are not permanently stored on-chain.

- Arbitrator decisions depend on the configured arbitrator.
- Production deployment would require additional security auditing.
- Real-money use would require additional legal, security and compliance considerations.

20. Future Improvements

- Multiple escrow contract support
- Multiple arbitrators
- Notification system
- Improved mobile responsiveness
- Professional smart contract security audit

21. Screenshot Documentation

Hardhat test execution – 11 passing tests

```

test('freelanceEscrow.test.js', { timeout: 10000 }, function () {
  describe('freelanceEscrow', function () {
    it('should allow arbitrator to resolve dispute', async function () {
      const escrow = await Escrow.deploy({
        from: owner,
        args: [
          0,
          freelancerAmount,
          clientAmount
        ]
      });

      const milestone = await escrow.getMilestone(0);
      expect(milestone).to.be.equal(0);
    });
  });
});

```

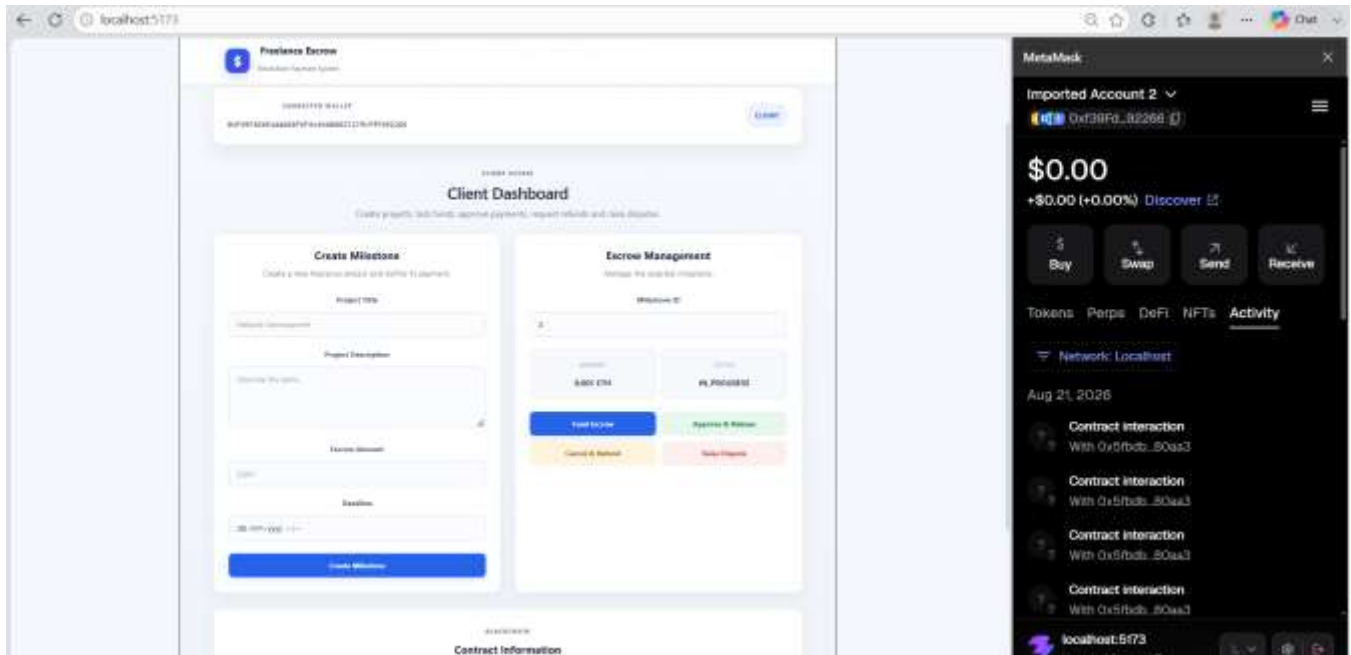
freelanceEscrow

- ✓ should deploy with correct rules (20ms)
- ✓ should create a milestone
- ✓ should fund milestone
- ✓ should reject incorrect funding amount (10ms)
- ✓ should allow freelancer to start work
- ✓ should reject unauthorized startWork
- ✓ should allow freelancer to submit work
- ✓ should release payment
- ✓ should prevent double payment
- ✓ should allow a dispute
- ✓ should allow arbitrator to resolve dispute (10ms)

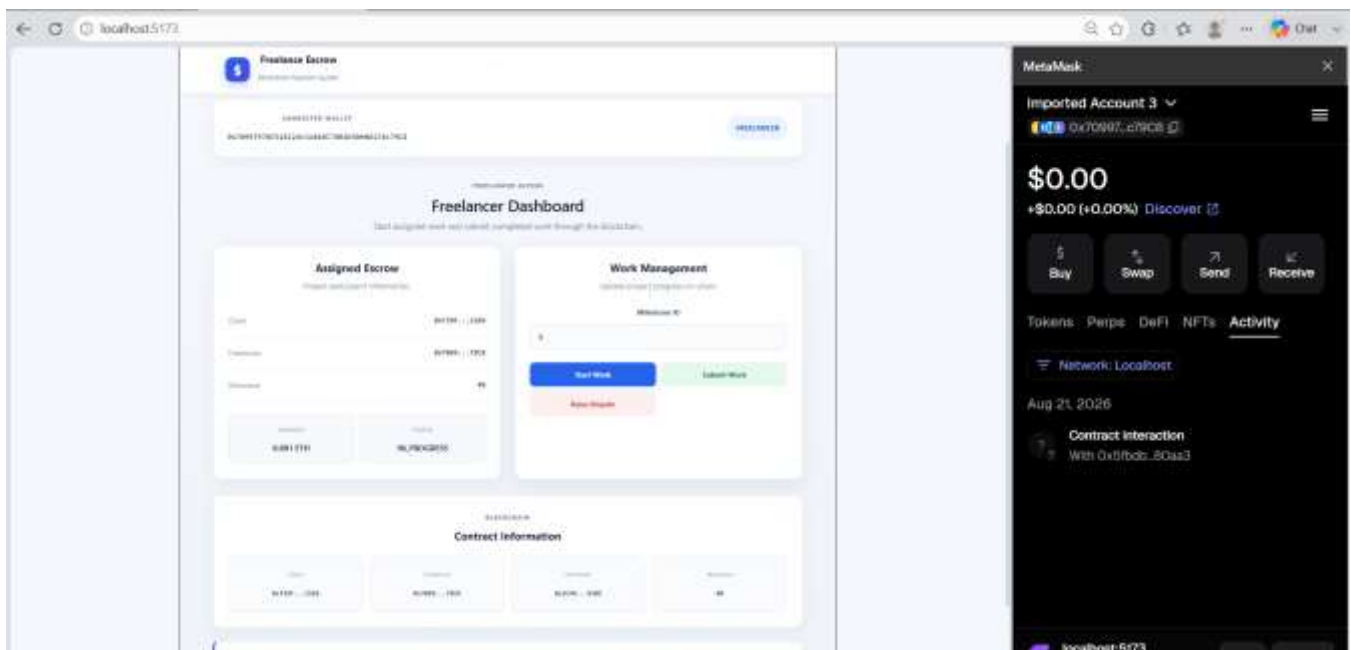
11 passing (2s)

%PS C:\Users\HP\Blockchain-based-Smart-Contract-Freelance-Escrow

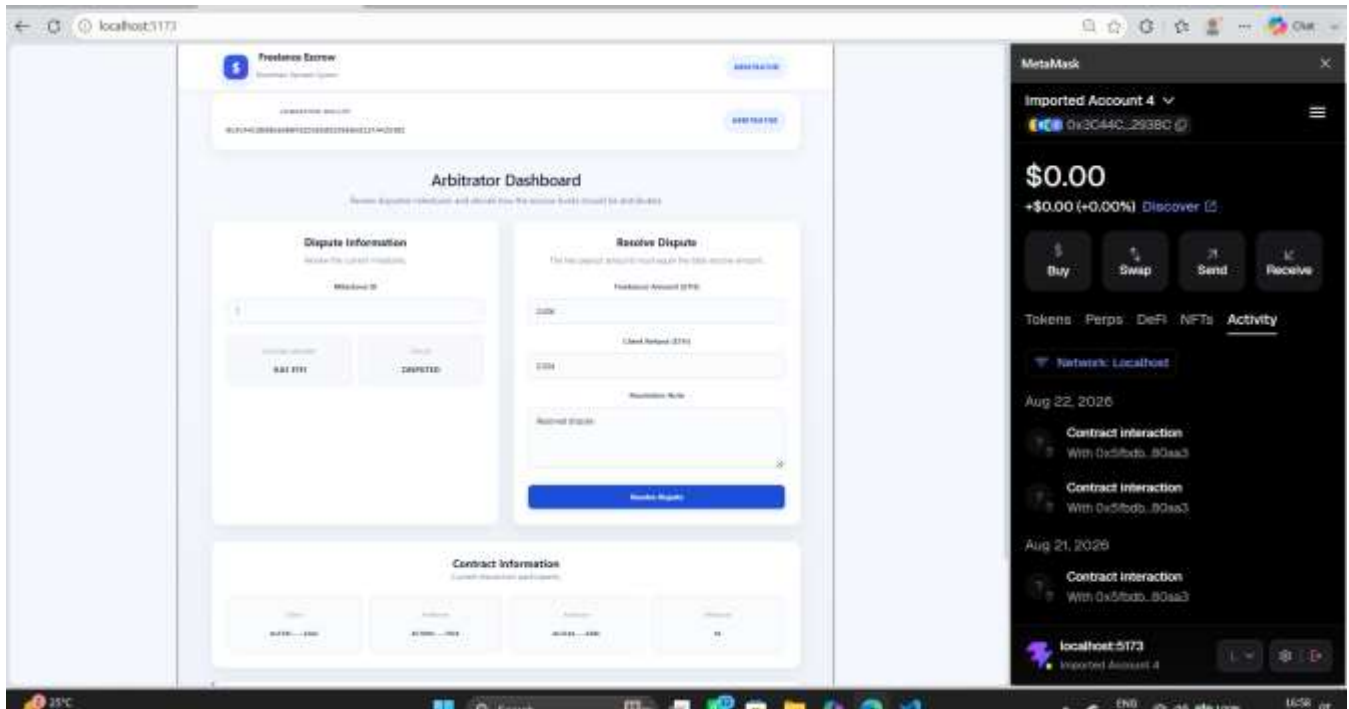
Client Dashboard – milestone creation and custom amount



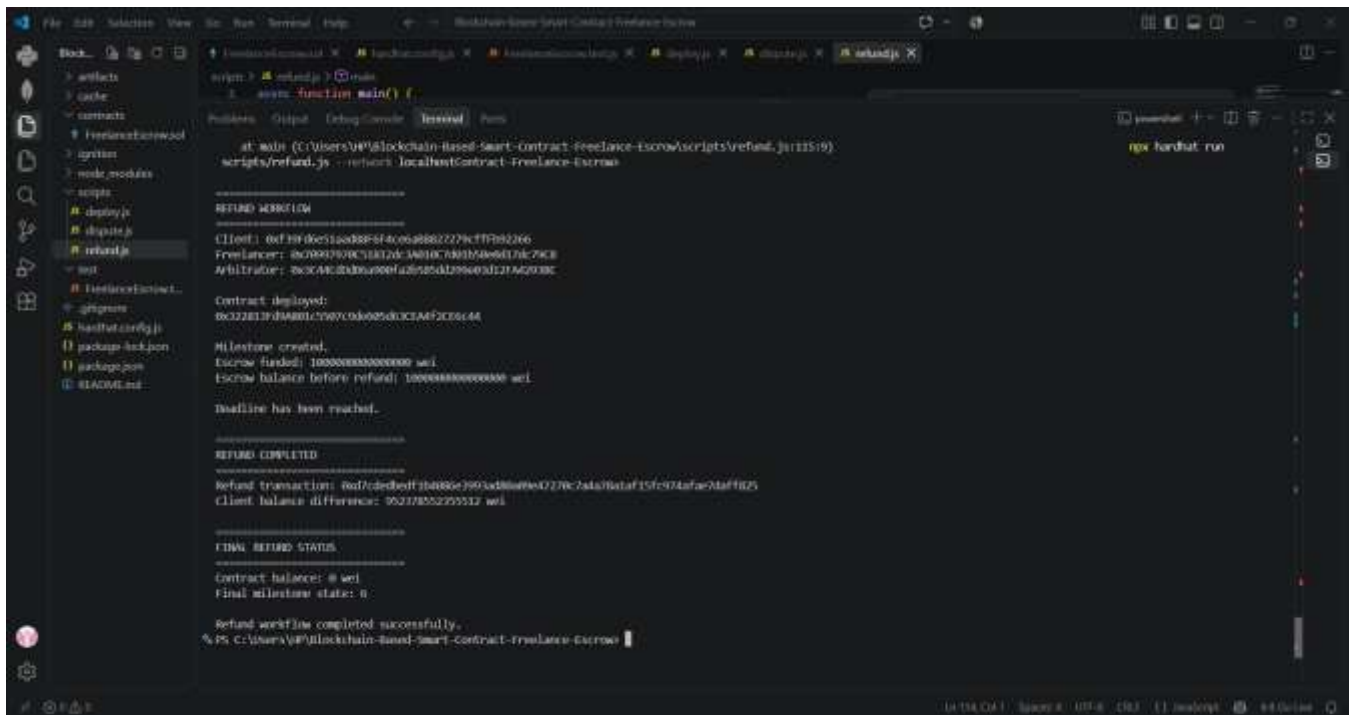
Freelancer Dashboard – assigned work and submission controls



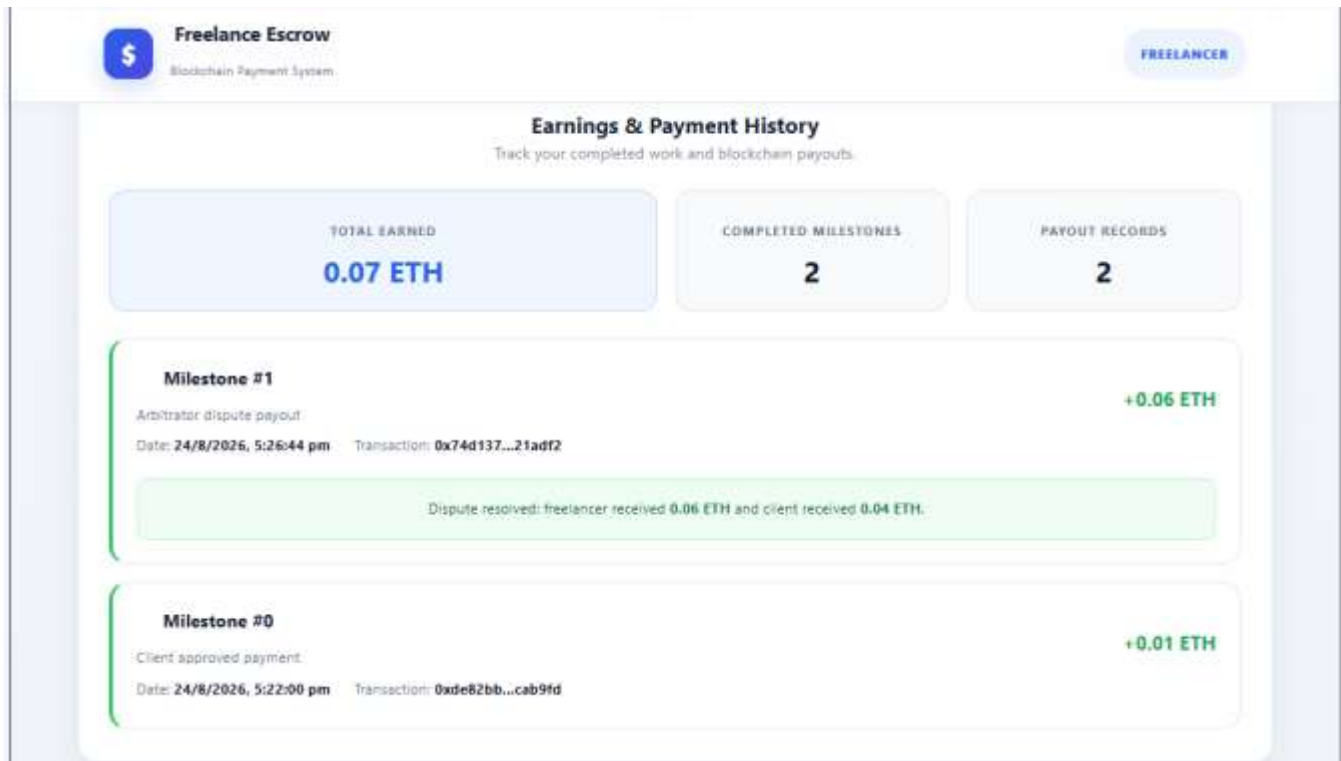
Arbitrator Dashboard – disputed milestone and resolution form



Refund Workflow – successful refund



History and Earnings – client milestone history and freelancer earnings/payment history



22. Advantages

- Transparent payment workflow
- Programmable escrow rules
- Custom milestone values
- Visible milestone states
- Structured dispute mechanism
- Role-specific access
- MetaMask transaction confirmation
- Safe local demonstration without real funds

23. Conclusion

The Blockchain-Based Freelance Payment Escrow DApp demonstrates how blockchain smart contracts can provide a transparent and programmable alternative to centralized freelance payment workflows. The system combines milestone-based payments, custom ETH funding, work progression, payment release, refunds, disputes, and arbitrator settlement with role-specific dashboards and transaction visibility.

The project therefore provides a practical educational demonstration of Web3 escrow architecture and the integration of Solidity, Hardhat, Ethers.js, React, and MetaMask.